

Dev Workflows

Git Workflows · Cycle Agile/Scrum · CI/CD

Concepts fondamentaux du developpement professionnel

Ce module couvre trois piliers du developpement logiciel professionnel : comment organiser le travail sur Git (branches, PRs, strategies), comment structurer un cycle de developpement (Agile/Scrum), et comment automatiser la livraison de code (CI/CD). Ces trois pratiques fonctionnent ensemble et definissent le flux de travail de la majorite des equipes tech.

PARTIE 1

Git Workflows

1. Pourquoi un workflow Git ?

Git permet de versionner le code, mais sans convention collective, plusieurs developpeurs qui travaillent en parallele finissent par creer des conflits, ecraser le travail des autres ou rendre l'historique illisible. Un **Git workflow** est un ensemble de regles sur comment creer des branches, nommer les commits, et fusionner le code.

2. Les trois strategies principales

2.1 GitFlow

GitFlow definit des branches avec des roles precis. C'est la strategie la plus structuree, adaptee aux projets avec des releases planifiees.

Branche	Role	Duree de vie
main	Code en production — TOUJOURS stable	Permanente
develop	Integration de toutes les features en cours	Permanente
feature/xxx	Developpement d'une nouvelle fonctionnalite	Temporaire (1 feature)
release/x.x.x	Preparation d'une version (tests, bugfixes finaux)	Temporaire (1 release)
hotfix/xxx	Correctif urgent directement sur main	Temporaire (1 fix)

```
# Creer une feature branch git checkout develop git checkout -b  
feature/US-12-resoudre-conversation # Travailler, committer... git commit -m  
'feat(US-12): bouton resoudre dans header' # Fusionner dans develop via Pull Request git
```

```
checkout develop git merge --no-ff feature/US-12-resoudre-conversation git branch -d feature/US-12-resoudre-conversation # Creer une release git checkout -b release/1.2.0 # tests, corrections mineures... git checkout main && git merge --no-ff release/1.2.0 git tag -a v1.2.0 -m 'Release 1.2.0'
```

2.2 Trunk-Based Development (TBD)

Dans le TBD, tous les developpeurs travaillent sur une seule branche principale (main/trunk) et mergent plusieurs fois par jour. Les features incompletes sont cachees derriere des **feature flags**. C'est la strategie utilisee par Google, Facebook et la majorite des equipes pratiquant DevOps matur.

ATTENTION TBD necessite une suite de tests solide et un pipeline CI/CD rapide. Sans tests automatises, merger directement sur main est trop risque.

Critere	GitFlow	Trunk-Based
Complexite	Elevee — beaucoup de branches	Simple — une seule branche active
Vitesse de livraison	Lente — cycles de release	Rapide — continu
Adapte a	Releases planifiees (ex: apps mobiles)	Deploiement continu (ex: web SaaS)
Risque de conflits	Faible (branches isolees)	Eleve sans bonne discipline
Tests requis	Recommandes	Obligatoires

2.3 GitHub Flow (simplifie)

GitHub Flow est un compromis : une seule branche protegee (main), des branches de courte duree par feature, et des Pull Requests pour merger. C'est le workflow le plus utilise par les petites equipes et les projets open-source.

```
1. git checkout -b feature/mon-truc # creer une branche 2. git commit -m 'feat: ...' # committer regulierement 3. git push origin feature/mon-truc # pousser 4. Ouvrir une Pull Request sur GitHub # demander une review 5. Code review + approbation # validation par un pair 6. Merge dans main + suppression branch # livraison
```

3. Conventions de commit

Les **Conventional Commits** sont un standard pour nommer les commits. Ils permettent de generer automatiquement des changelogs et de comprendre l'historique sans lire le code.

```
<type>(<scope>): <description courte> feat(inbox): ajouter filtre par plateforme  
fix(webhook): corriger validation signature Facebook refactor(orders): extraire logique  
de notification test(auth): ajouter tests JWT expiration chore(deps): mettre a jour  
expo-router vers 6.0.24 docs: ajouter guide de deploiement perf(db): ajouter index sur  
tenant_id + created_at revert: annuler commit abc1234
```

Type	Quand l'utiliser
feat	Nouvelle fonctionnalite pour l'utilisateur
fix	Correction de bug
refactor	Reorganisation du code sans changement de comportement
test	Ajout ou modification de tests
chore	Taches de maintenance (deps, config, tooling)
docs	Documentation uniquement
perf	Amelioration de performance
ci	Changements dans les pipelines CI/CD

4. Pull Requests et Code Review

Une **Pull Request (PR)** est une demande de fusion de code. Elle permet a un pair de relire le code avant qu'il n'atteigne la branche principale. C'est le mecanisme central de controle qualite dans un workflow Git.

Anatomie d'une bonne PR

- **Titre clair** : suivre la convention de commit (feat/fix + scope + description)
- **Description** : pourquoi ce changement ? Quel probleme resout-il ? Lier la user story
- **Petite taille** : une PR = une fonctionnalite. Moins de 400 lignes idealement
- **Tests** : les nouveaux tests doivent passer, l'existant ne doit pas regreer
- **Screenshots/videos** : obligatoires pour les changements UI
- **Self-review** : relire sa propre PR avant de demander une review

Checklist de review

- La logique est correcte et complete
- Les cas limites sont geres (null, vide, erreur reseau...)
- Pas de secrets ou credentials dans le code
- Les requetes DB filtrent par tenant_id (securite multi-tenant)
- Le code est lisible sans commentaires excessifs

- Les tests couvrent le nouveau comportement

5. Qu'est-ce qu'Agile ?

Agile est une philosophie de développement qui favorise des livraisons courtes et fréquentes plutôt qu'un grand projet livré une seule fois. Elle repose sur 4 valeurs fondamentales du Manifeste Agile (2001) :

On valorise PLUS...	...que
Les individus et leurs interactions	Les processus et les outils
Des logiciels opérationnels	Une documentation exhaustive
La collaboration avec le client	La négociation contractuelle
L'adaptation au changement	Le suivi d'un plan

6. Scrum — Le framework le plus utilisé

Scrum organise le travail en **sprints** : des cycles courts de 1 à 4 semaines (généralement 2 semaines). À la fin de chaque sprint, une version fonctionnelle du produit est livrée. Scrum définit des rôles, des artefacts et des cérémonies.

Rôles

Rôle	Responsabilité
Product Owner (PO)	Définit les priorités, gère le backlog produit, représente les utilisateurs
Scrum Master	Facilite les cérémonies, supprime les blocages, protège l'équipe
Équipe de développement	Réalise le travail (développeurs, designers, QA...)

Artefacts

Product Backlog — Liste ordonnée de tout ce qu'on veut construire. Géré par le PO.

Sprint Backlog — Sous-ensemble du Product Backlog sélectionné pour le sprint en cours.

Increment — Le produit fonctionnel livré à la fin du sprint. Doit répondre à la Definition of Done.

Cérémonies

Cérémonie	Quand	Durée	Objectif
Sprint Planning	Début du sprint	2-4h	Sélectionner les stories, estimer, s'engager
Daily Standup	Chaque jour	15 min	Synchroniser : hier / aujourd'hui / blocages

Sprint Review	Fin du sprint	1-2h	Presenter l'increment aux parties prenantes
Sprint Retrospective	Apres la review	1-2h	Ameliorer le processus de l'equipe

7. User Stories

Une **User Story** decrit une fonctionnalite du point de vue de l'utilisateur. Elle suit un format standard et est completee par des criteres d'acceptation.

```
Format standard : En tant que <role>, je veux <action> afin de <benefice>. Exemples :
US-11 : En tant qu'agent, je veux filtrer l'inbox par plateforme afin de traiter
rapidement les messages WhatsApp en priorite. US-12 : En tant qu'agent, je veux marquer
une conversation comme resolue afin de savoir quelles conversations necessitent encore
mon attention.
```

Criteres d'acceptation (Definition of Done)

Chaque User Story doit avoir des criteres explicites qui definissent quand elle est terminee :

- Le code est ecrit et revu (PR approuvee)
- Les tests unitaires et d'integration passent
- Le comportement correspond au scenario decrit dans la story
- La fonctionnalite est deployee en staging et validee
- Aucune regression sur les fonctionnalites existantes

8. Estimation et velocity

L'estimation en Scrum utilise les **Story Points** : une unite relative de complexite, pas de temps. On utilise la suite de Fibonacci (1, 2, 3, 5, 8, 13, 21...) car plus une tache est grande, plus l'incertitude est grande.

Points	Complexite	Exemple typique
1	Trivial	Corriger un typo ou une couleur
2	Simple	Ajouter un champ a un formulaire existant
3	Modere	Creer une nouvelle route API avec validation
5	Complexe	Implementer un systeme de filtres avec etat
8	Tres complexe	Integration avec une API tierce (webhooks...)
13+	Trop grand — a decouper	Feature entiere (ex: systeme de notifs)

La **velocity** est le nombre de story points completes par sprint. Apres 3-4 sprints, la velocity se stabilise et permet de planifier les futures iterations.

9. CI — Integration Continue

La **CI (Continuous Integration)** est la pratique de merger frequemment le code de tous les developpeurs vers une branche partagee, et de valider automatiquement chaque merge via une suite de verifications automatiques.

Que fait un pipeline CI ?

- 1 Declenchement**
Chaque push ou Pull Request declenche le pipeline automatiquement
- 2 Installation des dependances**
npm install ou pip install...
- 3 Lint / Formatage**
Verifier que le code respecte les conventions (ESLint, Prettier...)
- 4 Compilation / Type check**
TypeScript : tsc --noEmit — zero erreur de type
- 5 Tests automatises**
Unit tests, integration tests, E2E tests
- 6 Rapport**
Le pipeline indique succes ou echec avec le detail des erreurs

Exemple de pipeline CI avec GitHub Actions (fichier .github/workflows/ci.yml) :

```
name: CI on: [push, pull_request] jobs: backend: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-node@v4 with: { node-version: '20' } - run: cd backend && npm ci - run: cd backend && npx tsc --noEmit - run: cd backend && npx vitest run frontend: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run: npm ci - run: npx tsc --noEmit
```

10. CD — Deploiement Continu

Le **CD (Continuous Delivery / Deployment)** etend la CI jusqu'au deploiement. Il existe deux niveaux :

	Continuous Delivery	Continuous Deployment
Definition	Le code peut etre deploye a tout moment	Le deploiement est automatique apres chaque CI verte
Validation humaine	Oui — un humain approuve le deploiement	Non — tout est automatique
Risque	Faible — contr. manuel conserve	Plus eleve — necessite tests excellents
Adapte a	La plupart des equipes	Equipes tres matures avec haute couverture de tests

11. Environnements de deploiement

Un pipeline CI/CD mature distingue plusieurs environnements progressifs pour valider le code avant qu'il atteigne les utilisateurs reels.

Environnement	Objectif	Donnees	Declenche par
Development (local)	Developpement actif du code	Fictives / Seed	Le developpeur
Staging	Validation fonctionnelle pre-prod	Anonymisees	Merge sur develop/release
Production	Utilisateurs reels	Reelles	Merge sur main (apres validation staging)

REGLE Ne jamais tester en production. Staging doit etre le plus proche possible de prod (meme infra, memes variables d'env, meme version de DB). Les differences entre staging et prod sont la source numero 1 de bugs de deploiement.

12. Les types de tests dans un pipeline CI

La pyramide des tests definit la proportion ideale de chaque type de test. Plus un test est haut dans la pyramide, plus il est lent et couteux a maintenir.

Type	Vitesse	Cout	Ce qu'il teste	Exemple
Unit tests	Tres rapide	Faible	Une fonction / classe isolee	tester encryptToken()
Integration tests	Rapide	Moyen	Plusieurs composants ensemble	tester une route API avec DB
E2E tests	Lent	Eleve	Le flux complet utilisateur	simuler un achat complet
Contract tests	Rapide	Moyen	Interface entre services	API respecte son schema

CONSEIL La regle 70/20/10 : 70% unit tests, 20% integration tests, 10% E2E. Un pipeline CI uniquement compose d'E2E tests sera trop lent pour etre utile.

13. Les trois pratiques reunies

Git Workflow, Agile/Scrum et CI/CD ne sont pas independants — ils forment un systeme :

```
Sprint Planning → PO priorise le backlog → Equipe selectionne les stories du sprint  
Developpement (pendant le sprint) → Chaque story = une branche feature (GitHub Flow) →  
Commits conventionnels tout au long → Push → CI se declenche → tests passent → PR  
ouverte → Code review → merge dans develop Fin de sprint → Sprint Review : demo de  
l'increment → CD : deploiement en staging pour validation → Si OK : deploiement en  
production → Retrospective : amelioration du processus Le cycle recommence (sprint  
suivant)
```

SYNTHESE Un sprint bien execute produit un increment deployable a chaque fin de cycle. CI/CD garantit que cet increment est toujours dans un etat sain. Git workflow garantit que le travail parallele reste organise et reversible.